Guía de pruebas QA

SecurePlan Mobile (Calendario) | Sprint final

Documento	Guía de pruebas QA del sprint final
Dirigido a	José Manuel Mandujano
Entorno	Aplicación desplegada + APK + pruebas unitarias locales
Equipo de QA	Windows 10/11, 8 GB de RAM, Intel i3 de 13.ª generación
Versión	2.3
Fecha	5 de noviembre de 2026

La aplicación ya está desplegada. Las pruebas funcionales se hacen contra ese entorno con el APK y la versión web. Las unitarias corren en local con Node y mocks. El entorno local completo queda como apoyo para reproducir fallos.

Tiempo estimado: 90 a 120 minutos.
Entregable: informe con la tabla de resultados, capturas y los bugs encontrados.

1. Enfoque de las pruebas

Prueba principal: entorno desplegado	Apoyo: entorno local
Valida HTTPS, CORS, variables de producción y la base administrada.	Reproduce un bug sin tocar los datos reales.
Usa el APK publicado y la app web real.	Permite ver los logs del servidor.
No requiere Docker ni Node para las pruebas funcionales.	Requiere Docker y Node.
Escribe datos en la base real: usa cuentas QA.	Los datos son desechables.

No necesitas Expo Go. Expo Go sirve para desarrollo local; para QA usa el APK publicado y la URL web que te entregue el responsable.

Las pruebas contra el entorno desplegado crean usuarios y eventos reales. Usa correos con prefijo `qa+` y borra esos datos al terminar.

2. Entornos y artefactos

Artefacto	Dónde está	Cómo se accede
API	Servicio en la nube	URL que entrega el responsable.
Base de datos	PostgreSQL administrado en la nube	Solo a través de la API. No se conecta directo.
App web	Despliegue web	URL que entrega el responsable.
App Android	APK del repositorio	Instalar SecurePlan-Calendar-v1.0.1.apk.
Postman / Newman	Tu equipo	Importar la colección de qa/postman/.
Script de humo	Tu equipo	node qa/scripts/smoke.mjs.

Las URLs no se publican en el repositorio. Pídelas al responsable del proyecto antes de empezar.

3. Qué es /health y para qué sirve

El endpoint `/health` es el chequeo de salud de la API. Responde sin autenticación y confirma que el proceso está arriba y que la URL responde por HTTPS. Es la primera prueba de toda la sesión.

```
curl https://<api-desplegada>/health
```

La respuesta esperada es:

```
{"status":"ok"}
```

Sí confirma	No confirma
Que la API está arriba y responde.	Que la base de datos responda.
Que la URL y el certificado HTTPS funcionan.	Que el esquema tenga las tablas.
Que el servicio está en línea.	Que el registro o el CRUD funcionen.

Si `/health` responde ok pero el registro devuelve 500, el problema está en la conexión a la base o en el esquema, no en la API. El proveedor usa este endpoint como Health Check: si falla, reinicia el servicio. Después de un periodo de inactividad la primera petición puede tardar; espera unos segundos y reintenta.

4. Pruebas automatizadas

Ejecuta las dos contra la API desplegada y guarda la salida para el informe.

4.1 Script de humo (35 casos)

```
QA_BASE_URL=https://<api-desplegada> QA_ALLOW_REMOTE=1 node qa/scripts/smoke.mjs
```

El script crea usuarios y eventos reales, así que exige `QA_ALLOW_REMOTE=1` cuando el destino no es local. Sin esa variable se detiene para no ensuciar la base por accidente. Crea dos usuarios `qa+...@example.com`, recorre todo el flujo y termina con `35/35 pruebas OK`.

Bloque	Qué valida
Salud	El endpoint /health responde ok.
Autenticación	Registro, duplicado, clave corta, correo inválido, login, me y tokens inválidos.
Eventos	CRUD, campos extra, orden de horas, fecha, categoría e ids inválidos.
IDOR	Un segundo usuario recibe 404 al leer, editar o borrar un evento ajeno.
Exportación	JSON y CSV con solo los eventos del dueño, formato inválido y sin token.

Bloque	Qué valida
Cierre	Borrado 204, segundo borrado 404 y ruta inexistente 404.

Si el script falla en la primera petición, revisa la sección 3 y la sección 11.

4.2 Colección de Postman

Importa estos dos archivos:

```
qa/postman/SecurePlan-QA.postman_collection.json
qa/postman/SecurePlan-QA.postman_environment.json
```

Abre el entorno SecurePlan QA Local y cambia `baseUrl` por la URL de la API desplegada. Ejecuta las carpetas en orden: Salud, Autenticación, Eventos, Exportación, Seguridad.

La colección guarda el token, el correo y el id del evento después de cada petición. Para el caso de IDOR necesitas un segundo usuario: regístralo en Postman, crea un evento con su token y pega el id en la variable `otherEventId`. La petición debe responder 404.

Sin abrir Postman:

```
npx newman run qa/postman/SecurePlan-QA.postman_collection.json \
  -e qa/postman/SecurePlan-QA.postman_environment.json \
  --env-var baseUrl=https://<api-desplegada>
```

4.3 Datos de prueba

La base desplegada es la real. No borres usuarios ni eventos que no hayas creado. Para las pruebas destructivas (IDOR, borrado, datos inválidos) pide al responsable una rama de la base o un servicio de staging. Si no existe, ejecútalas al final y limpia solo tus cuentas `qa+`.

5. Pruebas manuales

Hazlas con Postman, Insomnia o curl apuntando a la API desplegada. Anota el resultado de cada una en el informe.

ID	Caso	Cómo	Resultado esperado
A01	Registro válido	POST /api/auth/register con correo nuevo y clave de 8 o más.	201, token y usuario sin password.
A02	Registro duplicado	Repite el mismo correo.	409.
A03	Clave corta	Registro con password 123.	400 con lista de errores.
A04	Login correcto	POST /api/auth/login.	200 con token.
A05	Login incorrecto	Misma cuenta con otra clave.	401.
A06	Me sin token	GET /api/auth/me sin header Authorization.	401.
A07	Me con token	GET /api/auth/me con Bearer.	200 con el usuario.
E01	Crear evento	POST /api/events con todos los campos.	201, date AAAA-MM-DD y horas HH:MM.
E02	Campo extra	Envía userId en el cuerpo.	400 por .strict().
E03	Horas invertidas	endTime anterior a startTime.	400.
E04	Fecha inválida	date con formato 01-12-2026.	400.
E05	Categoría inválida	category fiesta.	400.
E06	Listar eventos	GET /api/events con el token del dueño.	200 con solo sus eventos.

ID	Caso	Cómo	Resultado esperado
E07	Leer evento ajeno	Usuario B pide el id de A.	404.
E08	Editar evento ajeno	Usuario B hace PUT sobre el id de A.	404.
E09	Borrar evento ajeno	Usuario B hace DELETE sobre el id de A.	404.
E10	Borrar evento propio	DELETE con el token del dueño.	204 sin cuerpo.
X01	Export JSON	GET /api/events/export?format=json.	200 application/json con arreglo.
X02	Export CSV	GET /api/events/export?format=csv.	200 text/csv con cabecera.
X03	Formato inválido	format=xml.	400.
S01	Token alterado	Cambia un carácter del token.	401.
S02	Ruta inexistente	GET /api/desconocido.	404.
S03	Texto con SQL	title con '; DROP TABLE events; --.	Se guarda como texto y la tabla sigue.

6. App móvil y app web

6.1 APK de Android

Instala `SecurePlan-Calendar-v1.0.1.apk` en un dispositivo Android. El APK ya apunta a la API desplegada, así que no hay que configurar nada: si el login funciona, la conexión está bien.

Caso móvil	Resultado esperado
Registro y login	Entra a la agenda y la sesión se conserva al cerrar la app.
Crear evento	Aparece en el día correcto, sin corrimiento de fecha.
Editar y borrar	Los cambios se reflejan al volver a la agenda.
Recordatorio	La notificación se programa con la hora local.
Cerrar sesión	Vuelve al login y el token se borra del almacenamiento seguro.
Token vencido	La app regresa al login sin mostrar el error crudo.

6.2 App web

Abre la URL web que te entregue el responsable y repite el registro, el login y el CRUD. El almacenamiento seguro y las notificaciones son APIs nativas: si en el navegador la sesión no persiste o el recordatorio no aparece, anótalo como limitación de la versión web y valida ese caso en el APK.

Expo Go no forma parte de estas pruebas. Si lo usas, es solo para depurar en local.

7. Pruebas unitarias y auditoría (Sprint 4)

Estas pruebas no usan la API desplegada ni la base de datos: corren con mocks. Sirven para demostrar la cobertura de autenticación y fechas, y la auditoría de dependencias. La refactorización SOLID queda fuera de esta entrega: la arquitectura actual se mantiene. Tiempo estimado: 10 a 15 minutos.

Herramienta	Versión	Uso
Node.js	20 LTS (mínimo 18)	Ejecutar Jest.
npm	10 o superior	Instalar dependencias y correr scripts.
Git	Cualquiera	Clonar el repositorio.
Terminal	Cualquiera	Ejecutar los comandos.

No necesitas Docker, PostgreSQL, Android SDK ni la API desplegada para esta sección.

7.1 Instalación y ejecución

```
cd backend && npm ci
cd ../frontend && npm ci
```

Desde la raíz del repositorio, la forma corta:

```
bash qa/scripts/run-unit-tests.sh
```

O por separado:

```
cd backend && npm test -- --coverage
cd frontend && npm test -- --coverage
```

El script guarda los logs en `qa/sprint-4/evidencias/`.

7.2 Resultado esperado

Suite	Pruebas	Cobertura
Backend	40	auth.controller, user.model, token.service, auth.middleware y event.schema al 100%.
Frontend	17	date.js al 100%.
Total	57	Todas en verde.

La cobertura global del backend ronda el 54% porque incluye eventos y exportación, fuera de este alcance. Lo que Sprint 4 pide, autenticación y fechas, queda al 100%.

7.3 Auditoría de dependencias

```
bash qa/scripts/run-audit.sh
```

El backend de producción queda sin vulnerabilidades. El frontend mantiene 14 hallazgos de producción, 4 altas, todos transitivos de Expo. No ejecutes `npm audit fix --force`: baja Expo a la 52 y rompe la app. El detalle está en `qa/sprint-4/auditoria-dependencias.md`.

7.4 Qué significa "ejecutadas en vivo"

Las unitarias se muestran en la terminal con las 57 en verde. Las funcionales se muestran con el script de humo contra la API desplegada. Adjunta una captura de ambas ejecuciones.

8. Equipo y precauciones en Windows

El equipo de QA es una laptop Lenovo con Windows, 8 GB de RAM y un Intel Core i3 de 13.ª generación. Alcanza para toda la guía: la laptop no levanta la base de datos ni el backend, solo ejecuta Node y hace peticiones al entorno desplegado.

8.1 Requisitos del equipo

Recurso	Mínimo	Recomendado
RAM	8 GB	8 GB o más.
CPU	Intel i3 de 13.ª generación	i5 o Ryzen 5.
Disco libre	5 GB	8 GB.
Sistema	Windows 10 u 11	Windows 11.
Internet	Sí	Conexión estable.

No necesitas Docker, PostgreSQL, Android Studio ni emulador. El APK se prueba en un teléfono Android real.

8.2 Herramientas en Windows

Herramienta	Versión	Nota
Node.js	20 LTS	Evita Node 22 o 24: bcrypt y jest-expo pueden fallar.
Git para Windows	Última	Incluye Git Bash para los scripts .sh.
npm	Con Node	No instales otro gestor.
Teléfono Android	Android 8 o superior	Para el APK. No uses emulador.
Postman o Newman	Opcional	Newman consume menos RAM.

8.3 Instalación y rutas

Clona el repositorio en `C:\qa\Calendario`. Una ruta corta evita el límite de 260 caracteres y los bloqueos de OneDrive.

Abre Git Bash para los scripts `.sh`. PowerShell no ejecuta bash.

Si PowerShell bloquea `npm.ps1`, usa Git Bash o `npm.cmd`. No cambies la política de ejecución sin permiso del responsable.

No trabajes dentro de OneDrive: la sincronización bloquea `node_modules` y vuelve lento el `npm ci`.

Verifica que la fecha y la hora de Windows sean correctas. Un reloj desfasado invalida los JWT.

8.4 Precauciones

Precaución	Por qué
No ejecutes Docker ni un emulador de Android.	No hacen falta y consumen la RAM que necesita Jest.
Cierra Chrome, Teams y OneDrive durante el npm ci del frontend.	Es el paso más pesado: instala las dependencias de Expo.
Mantén al menos 5 GB libres.	node_modules del frontend ocupa entre 1.5 y 2.5 GB.
No uses Node 22 o 24.	bcrypt y jest-expo pueden fallar; usa Node 20 LTS.
No ejecutes npm audit fix ni --force.	Cambia dependencias y en el frontend baja Expo a la 52.
No uses cuentas reales.	El script de humo crea usuarios y eventos reales.

Precaución	Por qué
No borres datos que no empiecen con qa+.	La base desplegada es la de producción.
No compartas tokens ni capturas con Authorization.	Un token da acceso a la cuenta.
Espera la primera petición.	El servicio puede tardar en despertar; reintenta.
Permite la carpeta del proyecto en Defender.	El antivirus puede ralentizar npm ci; no desactives la protección.

8.5 Errores típicos en Windows

Error	Solución
npm.ps1 cannot be loaded	Usa Git Bash o npm.cmd.
bcrypt no compila	Instala Node 20 LTS y repite npm ci.
ENAMETOOLONG	Mueve el repo a C:\qa\Calendario.
EPERM o archivo bloqueado	Cierra OneDrive y VS Code, y reintenta.
bash no se reconoce	Abre Git Bash.
QA_ALLOW_REMOTE no se aplica	En PowerShell usa $env:QA_ALLOW_REMOTE="1".
No tests found	Ejecuta desde backend o frontend, no desde la raíz.

8.6 Dependencias después de clonar

El repositorio no trae `node_modules`. Después de clonar, instala las dependencias de cada carpeta con `npm ci`. No necesitas paquetes globales aparte de Node, npm y Git.

```
cd Calendario
cd backend && npm ci
cd ../frontend && npm ci
```

Carpeta	Comando	Qué instala
backend/	npm ci	express, pg, bcrypt, jsonwebtoken, cors, dotenv y zod. Para pruebas: jest y supertest.
frontend/	npm ci	expo, react, react-native, navegación, axios, secure-store y notificaciones. Para pruebas: jest-expo y eslint.
qa/	Nada	Los scripts usan Node y fetch; no tienen package.json.

En disco, el backend ocupa unos 70 MB y el frontend unos 1.6 GB después de instalar. Ejecuta `npm ci` desde cada carpeta, no desde la raíz.

Solo si vas a correr el backend en local, copia `backend/.env.example` a `backend/.env` y ajusta la conexión a PostgreSQL y `JWT_SECRET`. Para el frontend, copia `frontend/.env.example` a `frontend/.env` y define `EXPO_PUBLIC_API_URL` con la URL que te entregue el responsable. Ningún `.env` se versiona.

Herramienta	Cuándo instalarla
Docker Desktop	Solo si levantas PostgreSQL en local. Para QA contra la API desplegada no se usa.
Postman o Newman	Para ejecutar la colección. Opcional; el script de humo no lo necesita.
Expo Go	Solo para depurar el frontend en local. El APK no lo necesita.
JDK 17 y Android SDK	Solo para compilar el APK. No se instalan para QA.

No ejecutes `npm audit fix` ni `npm install --force`. Si `npm ci` falla por el lock, avisa al responsable antes de cambiar dependencias.

9. Evidencia y reporte de bugs

Guarda cada captura con el id del caso: `A01-registro.png`, `E07-idor.png`. Adjunta la salida del script y el resultado del Collection Runner.

Campo del bug	Qué escribir
Título	Acción y resultado en una línea.
Severidad	Bloqueante, alta, media o baja.
Entorno	Desplegado, navegador o modelo del teléfono, versión del APK.
Pasos	Lista numerada que cualquiera pueda repetir.
Esperado	Qué debía pasar.
Obtenido	Qué pasó, con el código HTTP o el mensaje.
Evidencia	Captura, log o salida de consola.

Severidad: bloqueante si impide usar la app; alta si rompe una función principal o expone datos de otro usuario; media si hay un error visible con solución alternativa; baja si es cosmético.

10. Limpieza de los datos de prueba

El script borra los eventos que crea, pero deja los usuarios `qa+...@example.com`. Si tienes acceso a la base, ejecuta:

```
psql "$DATABASE_URL" -f qa/scripts/cleanup.sql
```

Si no tienes acceso, pide al responsable que ejecute ese script. Nunca borres usuarios ni eventos que no lleven el prefijo de QA.

11. Problemas comunes en el entorno desplegado

Síntoma	Qué revisar
502, 503 o timeout en la primera petición	El servicio estaba inactivo. Espera unos segundos y reintenta.
/health responde ok pero registro da 500	Conexión a la base o esquema. Repórtalo con la hora exacta.
Todas las rutas dan 401	El token venció. Vuelve a hacer login.
Todas las rutas dan 404	La URL base está mal o le sobra una ruta.
Error de CORS en la app web	Configuración de CORS en la API. Repórtalo.
El APK no inicia sesión	El APK puede apuntar a una API vieja. Reporta la versión.
La app web no guarda la sesión	SecureStore no aplica igual en web. Valida en el APK.

12. Criterios de aceptación

Marca cada punto antes de entregar el informe.

- [ ] GET /health responde 200 con status ok en la API desplegada.
- [ ] El script de humo termina con 35/35 pruebas OK.
- [ ] La colección de Postman corre sin fallos, incluido el caso IDOR.
- [ ] Registro y login funcionan desde el APK.

[ ] Registro y login funcionan desde la app web.

[ ] Crear, editar y borrar eventos se refleja en la agenda.

[ ] Un usuario no puede leer, editar ni borrar eventos de otro.

[ ] Las validaciones rechazan fechas, horas y categorías inválidas.

[ ] Las respuestas nunca incluyen el campo password.

[ ] La exportación JSON y CSV contiene solo los eventos del usuario.

[ ] Las 57 pruebas unitarias pasan en backend y frontend.

[ ] La cobertura de autenticación y fechas llega al 100%.

[ ] npm audit --omit=dev del backend no reporta vulnerabilidades de producción.

[ ] Los hallazgos del frontend quedaron documentados sin aplicar --force.

[ ] Los usuarios qa+ creados durante la sesión quedaron borrados.

[ ] El informe incluye resultados, capturas y bugs con severidad.

13. Entrega

Entrega el informe al responsable del proyecto con la tabla de resultados, las capturas y la lista de bugs. Si no hay bugs bloqueantes, el sprint se acepta. El cierre del sprint y las evidencias están en `qa/sprint-4/cierre-sprint.md`.

Responsable QA	José Manuel Mandujano
Fecha de ejecución	
Resultado	Aprobado / Con observaciones
Firma	

Apéndice A. Pruebas en local (opcional)

Usa el entorno local solo para reproducir un fallo, ver logs o probar un cambio antes de desplegarlo. No es la vía principal de QA.

```
git clone <URL-del-repositorio>
cd Calendario_Practica/backend
npm install
cp .env.example .env
docker compose up -d
npm run dev
```

Con la API local arriba, el script corre sin la variable de seguridad: `node qa/scripts/smoke.mjs`. Para la app, `EXPO_PUBLIC_API_URL=http://<IP-local>:4000 npx expo start` y Expo Go en el mismo Wi-Fi. En el emulador de Android usa `http://10.0.2.2:4000`.